

MOCKITO HANDS-ON EXERCISES

EXERCISE 1: MOCKING AND STUBBING

ExternalApi.java

```
public interface ExternalApi {  
    String getData();  
}
```

MyService.java

```
public class MyService {  
    ExternalApi api;  
    public MyService(ExternalApi api) {  
        this.api = api;  
    }  
    public String fetchData() {  
        return api.getData();  
    }  
}
```

MyServiceTest.java

```
import org.junit.Test;

import org.mockito.Mockito;

import static org.mockito.Mockito.*;

import static org.junit.Assert.*;

public class MyServiceTest {

    @Test

    public void testExternalApi() {

        // Arrange - Create mock

        ExternalApi mockApi = Mockito.mock(ExternalApi.class);

        // Stub - define what mock returns

        when(mockApi.getData()).thenReturn("Mock Data");

        // Act

        MyService service = new MyService(mockApi);

        String result = service.fetchData();

        // Assert

        assertEquals("Mock Data", result);

    }

}
```

OUTPUT

The screenshot shows an IDE window for a project named 'WeekTwoTDD'. The 'Project' view on the left shows a directory structure with 'src' containing 'main' and 'test' folders. The 'test' folder contains 'java' and 'resources' subfolders. The 'java' folder contains 'AAAPatternTest', 'AssertionsTest', 'CalculatorTest', and 'MyServiceTest'. The 'MyServiceTest' class is selected. The 'Run' view at the bottom shows the execution of 'MyServiceTest' with the following output:

```
MyServiceTest 556 ms
testExternalApi 556 ms
1 test passed 1 test total, 556 ms
C:\Program Files\Java\jdk-23\bin\java.exe ...
Process finished with exit code 0
```

The code editor shows the following Java code for 'MyServiceTest.java':

```
6 public class MyServiceTest {
7     public void testExternalApi() {
8
9         ExternalApi mockApi = Mockito.mock(ExternalApi.class);
10
11         // Stub - define what mock returns
12         when(mockApi.getData()).thenReturn("Mock Data");
13
14         // Act
15         MyService service = new MyService(mockApi);
16         String result = service.fetchData();
17
18         // Assert
19         assertEquals("Mock Data", result);
20     }
21 }
22
23 }
```

EXERCISE 2: VERIFYING INTERACTIONS

MyServiceTest.java

```
import org.junit.Test;

import org.mockito.Mockito;

import static org.mockito.Mockito.*;

import static org.junit.Assert.*;

public class MyServiceTest {

    @Test

    public void testExternalApi() {

        ExternalApi mockApi = Mockito.mock(ExternalApi.class);

        when(mockApi.getData()).thenReturn("Mock Data");

        MyService service = new MyService(mockApi);

        String result = service.fetchData();

        assertEquals("Mock Data", result);

    }

    @Test

    public void testVerifyInteraction() {

        ExternalApi mockApi = Mockito.mock(ExternalApi.class);

        MyService service = new MyService(mockApi);

        service.fetchData();

        verify(mockApi).getData();

    }

}
```

OUTPUT

